Progetto

Introduzione

Il sistema possiamo classificarlo come un **Intrusion Prevention System (IPS) perimetrale a livello applicativo,** basato su un'architettura proxy *Store-and-Forward*. Nella sua attuale configurazione architetturale, viene analizzato specificamente **traffico SMTP (Simple Mail Transfer Protocol) incapsulato in TLS**.

Il sistema opera un'ispezione profonda dei pacchetti (Deep Packet Inspection) integrando l'analisi crittografica delle sessioni TLS con una pipeline di Intelligenza Artificiale a doppio stadio, al fine di rilevare e mitigare in tempo reale minacce di rete automatizzate e campagne di ingegneria sociale generate sinteticamente.

Nel dettaglio il sistema vuole essere un sistema di detection che sfrutta sia la logica di Anomaly Detection basata su modelli di Machine Learning che su firme o euristiche sfruttando lo scambio delle informazioni durante una sessione SSL.

Obiettivi

- **Deep Packet Inspection (DPI) su traffico TLS:** Implementazione di una terminazione TLS per l'ispezione dei payload cifrati.
- **Hybrid Detection:** Correlazione tra analisi dei metadati (**JA3 Fingerprinting**) e analisi semantica (**AI**).
- **Security by Design:** Utilizzo di un'architettura containerizzata per proteggere il punto di decifratura.
- **Scalabilità e Performance:** Esecuzione di inferenza neurale su CPU per scenari di traffico moderato.

Progettazione e Ingestione (far funzionare il flusso base)

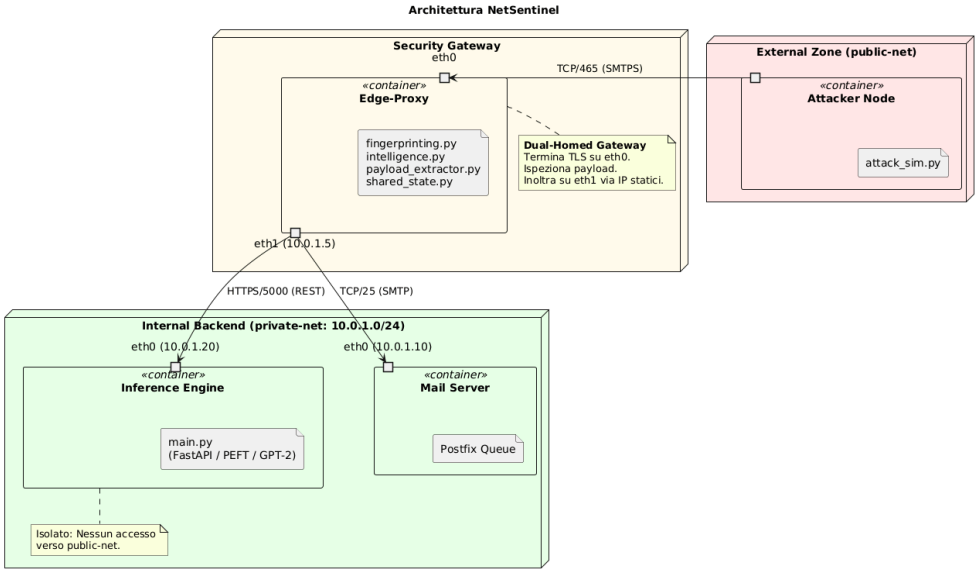
Struttura:

```
NetSentinel/
├── docker-compose.yml
├── analysis/ # Nodo di inferenza
│   ├── archive/
│   ├── fine_tuned_ips_adapter/ # Pesi del modello in seguito al fine-tuning
│   ├── Dockerfile
│   ├── main.py
│   └── requirements.txt
├── attacker/ # Nodo di attacco
│   ├── Dockerfile
│   ├── attack_sim.py
```

```
│   └── test_MIME.py
│
├── mail/ # Nodo mail-server
│   └── Dockerfile
├── mitmproxy_certs/ # Certificati di mitmproxy
│
└── proxy/ # Nodo edge-proxy
    ├── Dockerfile
    ├── fingerprinting.py
    ├── intelligence.py
    ├── payload_extractor.py
    └── shared_state.py
```

Andiamo a definire l'architettura del sistema.

L'ambiente di simulazione implementa una **Network Segregation** tramite l'uso di bridge network multipli.



Il sistema si suddivide in due segmenti di rete isolati tramite driver bridge di Docker:

A. Public-Net (Zona Untrusted)

Lo scopo è quello di simulare la rete esterna o una DMZ esposta ai tentativi di attacco.

- **Componenti**: `attacker-node`.

  Nessun instradamento diretto verso il backend. La risoluzione dei nomi (DNS) è limitata ai nodi presenti in questa sottorete.

B. Private-Net (Zona Trusted - 10.0.1.0/24)

- **Scopo**: Isolare l'intelligenza decisionale e il server di destinazione.
- **Componenti**: `inference-engine`, `mail-server`.

  Rete totalmente "dark" rispetto all'esterno. L'accesso a Internet o ad altre reti è inibito per prevenire l'esfiltrazione di dati in caso di compromissione del server mail.

L'invio dei dati all'interno della rete privata è stato effettuato garantendo la crittografia necessaria per rendere il sistema più sicuro.

L' `edge-proxy` funge da **Application Proxy** e costituisce il cuore dell'architettura.

- **Interfaccia Pubblica**: In ascolto sulla porta 465 (SMTPS), riceve il traffico cifrato dall'attaccante.
- **Interfaccia Privata (10.0.1.5)**: Utilizzata per dialogare con i servizi interni.

- **Ispezione e Terminazione**: Il proxy *termina la sessione TLS*, *decifra il payload* in memoria e *lo sottopone all'analisi sia in locale*, applicando una **logica di IPS basato su firma**, e in secondo luogo, una **logica IDS basata su anomaly detection** *tramite l'ausilio di modelli di Machine Learning*.

  Il traffico non viene mai instradato a livello di pacchetto (L3), ma viene ricostruito e reinviato a livello applicativo (L7) solo se validato dal nodo AI.

Flusso di esecuzione

1. **Intercettazione:** Il traffico SMTP viene intercettato. Il fingerprinting JA3 avviene durante l'handshake TLS e il dato viene propagato in memoria condivisa.
2. **Manipolazione (Store-and-Forward)**: I pacchetti TCP contenenti il payload della mail vengono messi in coda. A ricezione ultimata, il contenuto viene inviato all'analisi e, in base alla risposta, i byte dell'header `Subject:` vengono riscritti al volo prima dell'inoltro.

   > **NOTA:** questo approccio di notifica andrebbe poi rivisto per quanto riguarda l'integrazione con DKIM.

3. **Sicurezza (Zero Trust)**: La comunicazione tra il proxy e il nodo di inferenza è segregata in una rete privata e protetta da un tunnel HTTPS. I certificati vengono generati dinamicamente all'avvio e montati tramite volumi condivisi Docker.
4. **Recapito (Mail Server)**: Postfix è configurato rigorosamente per accettare il traffico filtrato ed effettuare la consegna locale (Local Delivery Agent) alla casella corretta dell'utente, senza loop di routing o problemi di risoluzione DNS.

Network Map

Nodo	Rete	IP Statico	Porta Servizio	Funzione
`attacker-node`	`public-net`	Dinamico	-	Simulazione Attacco
`edge-proxy`	`public/private`	`10.0.1.5`	465	Ingestione / Ispezione
`inference-engine`	`private-net`	`10.0.1.20`	5000	Analisi Neurale
`mail-server`	`private-net`	`10.0.1.10`	25	Consegna Finale

Tecnologie implementate

È bene introdurre la serie di tecnologie utilizzate per la realizzazione del sistema di rilevazione, per ognuna di esse è stata fornita una descrizione più o meno dettagliata, coerente anche con l'uso che ne è stato fatto.

Docker

Docker offre la possibilità di impacchettare ed eseguire un'applicazione in un ambiente debolmente isolato chiamato **container**. L'isolamento e la sicurezza consentono di eseguire *più container contemporaneamente su un determinato host*.

I container sono leggeri e contengono tutto il necessario per eseguire l'applicazione, quindi non è necessario fare affidamento su ciò che è installato sull'host. È possibile condividere i container mentre si lavora e avere la certezza che tutti coloro con cui si condivide abbiano lo stesso container che funziona allo stesso modo.

Nel laboratorio, Docker non viene usato solo come gestore di container, ma come **emulatore di rete**.

Scapy

Scapy è una libreria Python per la manipolazione di pacchetti a basso livello. A differenza degli sniffer tradizionali (come Wireshark) che si limitano a visualizzare i dati, o dei tool di iniezione (come hping) che hanno opzioni rigide, Scapy permette di **costruire**, **decodificare**, **inviare** e **sniffare** pacchetti di quasi tutti i protocolli conosciuti.

Mentre **mitmproxy** lavora a livello applicativo (Layer 7), Scapy lo usiamo in particolare per 3 motivi:

1. **Pre-Decryption Fingerprinting:** Il JA3 si basa sui parametri inviati dall'attaccante nel pacchetto *Client Hello*. Poiché **mitmproxy** termina la connessione TLS per decifrarla, a volte "nasconde" i pacchetti grezzi dell'handshake. Con Scapy, possiamo sniffare l'interfaccia di rete e catturare il *Client Hello* originale **prima** che il proxy lo elabori.

2. **Validazione del Protocollo:** Un attaccante potrebbe tentare di inviare traffico non-SMTP sulla porta 465 per bypassare i controlli. Scapy ci permette di verificare se la struttura dei pacchetti TCP è coerente con una sessione TLS legittima prima ancora di avviare la decifratura.
3. **Estrazione Metadati di Basso Livello:**Possiamo estrarre informazioni che i proxy di alto livello ignorano, come il **TTL (Time To Live)** o la **Window Size** del pacchetto TCP dell'attaccante. Questi dati aiutano nel **OS Fingerprinting**: capire se l'attaccante sta usando Linux, Windows o uno script automatizzato.

JA3 Fingerprinting

JA3 è uno standard open source, creato originariamente da *Salesforce*, utilizzato per creare un'impronta digitale (fingerprint) dei client SSL/TLS, il suo scopo è identificare un'applicazione (che sia un browser legittimo, uno script o un malware) *in base al modo in cui avvia una connessione cifrata (* parametri del pacchetto *Client Hello*.), indipendentemente dall'indirizzo IP di destinazione, porte e i certificati X509.

Come sappiamo, per avviare una connessione SSL/TLS, un client invierà un pacchetto **SSL Client Hello** dopo aver eseguito l'handshake TCP a 3 vie. Questo pacchetto e il modo in cui viene generato *dipendono dai pacchetti e dai metodi utilizzati durante la creazione* dell'applicazione client.

Il server, se accetta connessioni SSL, risponderà con un pacchetto **SSL Server Hello** formulato in base alle librerie e alle configurazioni lato server, nonché ai dettagli nel pacchetto **SSL Client Hello**. Poiché **le negoziazioni SSL vengono trasmesse in chiaro**, è possibile identificare le applicazioni client utilizzando i dettagli nel pacchetto SSL Client Hello.

L'algoritmo JA3 estrae i valori decimali di **5 campi specifici** dal pacchetto Client Hello:

1. **Versione SSL**
2. Cifrari accettati (**Cipher Suites**)
3. Lista delle estensioni (**SSL Extensions**)
4. Curve Ellittiche (**Elliptic Curves**)
5. Formati dei punti delle Curve Ellittiche (**Elliptic Curve Point Formats**).

   Gli EC Formats non cambiano *quale* curva si usa (quello lo fanno i **Supported Groups**), ma definiscono il **linguaggio** con cui ci si scambia i punti di quella curva. È un dettaglio tecnico sottile, ma utilissimo per distinguere, ad esempio, un client Java da uno scritto in Go o Python.

Questi valori vengono concatenati in ordine, utilizzando una virgola (`,`) per separare i campi e un trattino (`-`) per separare i valori all'interno di un campo.

La stringa risultante viene poi convertita in un hash **MD5** a 32 caratteri. Questo hash è il **fingerprint JA3**.

Un aspetto critico nell'implementazione dell'algoritmo JA3 è la gestione dei valori **GREASE** (*Generate Random Extensions And Sustain Extensibility*). Introdotti da Google e formalizzati nella **RFC 8701**, questi valori sono parametri "fantoccio" inseriti casualmente nei campi delle Cipher Suites, delle Extension e dei Supported Groups per evitare che i server diventino troppo rigidi nell'accettare solo determinati formati.

Poiché questi valori cambiano a ogni nuova sessione TLS, se venissero inclusi nel calcolo, l'hash JA3 cambierebbe continuamente anche per la stessa identica applicazione. Per questo motivo, l'algoritmo JA3 deve applicare un filtraggio rigoroso: tutti i valori identificati come GREASE (riconoscibili perché seguono un pattern specifico, solitamente nella forma `0x?A?A`).

Una volta ottenuto l'**hash JA3**, possiamo utilizzarlo principalmente per la **difesa e l'analisi della sicurezza**, sfruttando il fatto che questa impronta rimane costante anche quando altri indicatori cambiano. Questo consente due applicazioni principali:

1. **Rilevamento di Malware:** Il vantaggio principale del JA3 è la sua capacità di identificare un'applicazione client indipendentemente dalla destinazione della connessione.

   Confrontando il JA3 ottenuto con liste di impronte note, possiamo identificare minacce specifiche come **Trickbot** o **Emotet**, o applicazioni come il client **Tor**, basandoci solo su *come* comunicano e non su *chi* contattano.

2. **Creazione di Whitelist:** In ambienti aziendali "blindati" o ad alta sicurezza, dove sono permesse solo poche applicazioni specifiche, il JA3 è uno strumento eccellente per il controllo.

```
https://github.com/salesforce/ja3
```

Nel nostro caso andremo a sfruttare l'hash di JA3 nel nodo di ingestione (**edge-proxy**) in modo che prenda decisioni autonome basate sulla criticità:

- **Categoria MALWARE**: Se l'hash JA3 è presente nel database di Abuse.ch, la connessione viene considerata compromessa al 100%. Il proxy può generare un **Alert ad alta priorità** per il SOC (Security Operations Center) e interrompere la catena di analisi (esegue un `TCP Kill` e invia un report di incidente asincrono al backend), *risparmiando token* e *tempo di calcolo sul nodo AI.*
- **Categoria AUTOMATION / UNKNOWN**: In questo caso, l'origine non è di per sé una prova di malignità. Lo script Python potrebbe essere un test autorizzato o una notifica legittima. Qui l'analisi del contenuto (Payload) diventa fondamentale per decidere se il messaggio è phishing o meno.

Mitmproxy

Mitmproxy è un set di strumenti che fornisce un proxy di intercettazione interattivo e compatibile con SSL/TLS per HTTP/1, HTTP/2 e WebSocket. Tra le caratteristiche messe a disposizione ci concentreremo su:

- Intercettazione di richieste e risposto HTTP e HTTPS, oltre alla capacità di modificarle al volo.
- Apporta modifiche tramite script al traffico HTTP utilizzando Python
- Modalità proxy inverso per inoltrare il traffico a un server specificato.
- I certificati SSL/TLS per l'intercettazione vengono generati al volo.

Quando parliamo di Mitmproxy di solito ci riferiamo a uno qualsiasi dei tre strumenti:

mitmproxy è un proxy di intercettazione interattivo, compatibile con SSL/TLS, con un'interfaccia console per HTTP/1, HTTP/2 e WebSocket.

mitmweb è un'interfaccia basata sul web per mitmproxy.

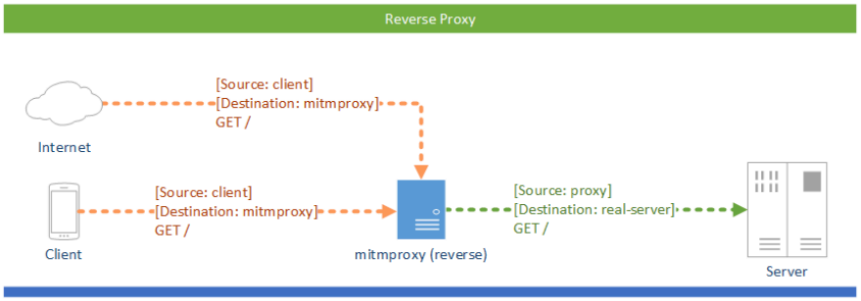
mitmdump è la versione a riga di comando di mitmproxy. Pensiamo a **tcpdump** per HTTP.

Mitmproxy può decifrare il traffico crittografato al volo, a condizione che il client si fidi dell'autorità di certificazione integrata in Mitmproxy. In genere, ciò significa che il certificato CA di Mitmproxy deve essere installato sul dispositivo client.

La prima volta che mitmproxy viene eseguito, vengono create le chiavi per un'autorità di certificazione (CA) nella directory config (`~/.mitmproxy` per impostazione predefinita). Questa CA viene utilizzata per la generazione al volo di certificati fittizi per ogni sito web visitato. Poiché il browser non considera attendibile la CA di mitmproxy fin dall'inizio, sarà necessario visualizzare un avviso relativo al certificato TLS su ogni dominio oppure installare il certificato CA una volta sola, in modo che sia attendibile.

Mitmproxy consente diverse modalità di proxy, nel nostro caso saremo interessati alla modalità **Reverse Proxy.** `mitmdump --mode reverse:https://example.com`

In modalità reverse proxy, mitmproxy agisce come un normale server. Le richieste dei client vengono inoltrate a un server di destinazione preconfigurato e le risposte vengono inoltrate al client:



Nonostante sia disponibile la possibilità di implementare il Reverse Proxy su tanti protocolli (HTTP o HTTPS, DNS, QUIC, TCP, TLS), si è scelto di operare a livello **TCP** (Layer 4) invece che **TLS** (Layer 7) per l'intercettazione iniziale del JA3, dettata dalla necessità di accedere ai dati "grezzi" prima che vengano alterati dal processo di decifratura.

Questo ci permette di guardare dentro il payload TCP prima che la macchina a stati TLS del proxy modifichi o risponda al client, garantendo che l'hash JA3 sia quello autentico del bot e non quello "rielaborato" dal proxy.

Postfix

Postfix è un **MTA (Mail Transfer Agent)**, ovvero il software responsabile dello smistamento e dell'invio della posta elettronica tra server attraverso il protocollo SMTP.

Il funzionamento interno di Postfix è basato su un'architettura a **componenti modulari**. Postfix utilizza diversi demoni specializzati che comunicano tra loro, garantendo sicurezza e resilienza.

Quando una mail arriva dall'esterno o da un client locale, viene presa in carico da uno dei componenti di "ingresso":

- **smtpd:** Gestisce le connessioni SMTP provenienti dalla rete. Nel nostro caso con il proxy.
- **pickup:** Raccoglie i messaggi generati localmente sulla macchina (es. tramite il comando `mail`).

Questi demoni passano il messaggio al processo **cleanup**, che si occupa di normalizzare gli header e preparare il messaggio per la coda.

Una volta elaborata, la mail viene scritta in un file nella directory `/var/spool/postfix/incoming`. Qui entra in gioco il cuore di Postfix: il **Queue Manager (qmgr)**.
Il `qmgr` è lo smistatore del sistema:

- Sposta i messaggi dalla coda `incoming` alla coda `active` quando il sistema è pronto a processarli.
- Gestisce la coda `deferred` per le mail che non è stato possibile consegnare immediatamente (es. server di destinazione temporaneamente offline).

Il `qmgr` contatta gli agenti di consegna (**Delivery Agents**) in base alla destinazione finale del messaggio:

- **smtp:** Se la mail è destinata a un server esterno (es. @gmail.com), il demone `smtp` agisce come client e la inoltra via rete.
- **local:** Se la mail è destinata a un utente reale del sistema Linux presente in `/etc/passwd`.
- **virtual:** Se si utilizza un database (MySQL/MariaDB) per gestire gli utenti (Virtual Users), questo componente si occupa di consegnare la mail nella directory corretta sul disco (es. `/var/vmail/dominio/utente`).

Quando Postfix deve decidere "dove" mettere la mail di un utente virtuale, non guarda i file di sistema ma interroga le **Lookup Tables.**
Queste tabelle sono definite nel file `main.cf`. Postfix esegue una query SQL in tempo reale per tradurre l'indirizzo email nel percorso fisico della cartella `Maildir` sul disco.

Python AI Inference Engine

L'idea alla base della realizzazione del sistema è concepita come diretta conseguenza del funzionamento dei **Language Model.**

Un modello linguistico è un sistema basato sul machine learning che ha l'obiettivo di predire le parole successive all'interno di una frase. Lo fa assegnando una probabilità a ciascuna possibile parola futura, oppure valutando la probabilità di un'intera sequenza di parole.

Per calcolare questa probabilità, bisognerebbe idealmente considerare tutta la frase che la precede. Tuttavia, poiché il linguaggio è estremamente creativo, molte sequenze esatte di parole non si ripetono mai, rendendo impossibile calcolarne le probabilità in modo diretto. Per aggirare il problema, introduciamo:

- **Gli N-grams**: Sequenze di *n* parole. Un *unigramma* analizza una parola singola, un *bigramma* coppie di parole (es. "L'acqua di"), un *trigramma* triplette, e così via.
- **L'Assunzione di Markov**: Invece di considerare l'intera storia di una frase, si approssima il contesto guardando solo le ultime *n-1* parole precedenti. In un modello a bigrammi, ad esempio, la probabilità della parola successiva dipende esclusivamente dalla parola immediatamente precedente.

Le probabilità vengono stimate tramite la **Stima di Massima Verosimiglianza (MLE).**

$$\log P(w_i|w_1, ..., w_{i-1})$$

La metrica standard per valutare un modello linguistico si chiama **perplessità (perplexity)**. Essa misura quanto il modello è "sorpreso" dai dati del test set: più bassa è la perplessità, maggiore è la probabilità che il modello ha assegnato al testo, indicando così che il modello ha previsto accuratamente le parole. Poiché un LLM tende a scegliere le parole con la probabilità più alta nell'albero decisionale, il testo generato da IA presenta tipicamente una **perplessità molto più bassa rispetto a un testo scritto da un umano**.

Possiamo rappresentare la perplessità da un punto di vista matematico:

$$PP = \exp\left(-\frac{1}{N}\sum_{i=1}^{N}\log P(w_i|w_1, ..., w_{i-1})\right)$$

Da questo concetto possiamo pensare di andare a verificare se un testo è generato da un LLM andando a valutare questo valore associato al contenuto. Un valore sotto una determinata soglia critica indicherà un testo con bassa varianza, tipico degli output sintetici.

La **Perplexity** valuta la predicibilità a livello micro-strutturale (il singolo token o la singola parola). Un classificatore basato esclusivamente su questo parametro è soggetto a due criticità statistiche che minano l'affidabilità di un sistema IPS:

- **Falsi Positivi su Testo Formale**.
- **Falsi Negativi per Adversarial Prompting:** Un attaccante consapevole della presenza di un IPS basato sulla Perplexity può istruire il Large Language Model a generare l'email di phishing utilizzando intenzionalmente vocaboli imprevedibili.

Per aggiungere un controllo a livello macro-strutturale (la frase e l'intero documento), aggiungiamo la **Burstiness.**

Un LLM a cui viene forzato l'uso di parole rare (innalzando la Perplexity) manterrà comunque l'impronta statistica del suo strato di *Self-Attention*: continuerà a generare frasi di lunghezza media costante, con strutture sintattiche omogenee e prive di reali fluttuazioni del ritmo. La Burstiness rimarrà inevitabilmente bassa.

Al contrario, un operatore umano che scrive un testo formale e prevedibile (bassa Perplexity) manterrà comunque la sua firma cognitiva inconscia: alternerà periodi principali molto lunghi a frasi subordinate più brevi o incisi, generando un'alta Burstiness.

Nel nostro caso andremo a effettuare quindi un **Double-Check**:

- **Classificatore Neurale (RoBERTa-base)**: Un modello sottoposto a fine-tuning specifico su dataset quadri-classe per identificare l'intento malevolo e i pattern semantici tipici degli LLM.
- **Analisi Statistica (GPT-2)**: Utilizzato per il calcolo della Perplexity, agendo come sensore di validazione per confermare o smentire l'origine artificiale del payload.

Componenti

Nodo edge-proxy

L' **Edge-Proxy** è il cuore operativo del sistema. Agisce come un'entità attiva che interrompe la catena di comunicazione per validare il contenuto.

È bene notare che l'Edge Proxy agisce come l'endpoint da contattare affinché sia in grado di poter interrompere il tunnel TLS su di esso e quindi, decifrare, leggere e poi aprire un secondo tunnel verso il mail server.

Specifiche Tecniche

- **Core Engine:** mitmproxy 12.2.1 (Rust-based TLS parser).
- **Networking:** Dual-homed (interfaccia su `public-net` e `private-net`).
- **Protocolli gestiti:** SMTP (porta 465) incapsulato in TLS.
- **Intelligence:** Integrazione con modulo locale `intelligence.py` e remoto `Inference Engine`.

```
edge-proxy:
    build:
      context: ./proxy
      dockerfile: Dockerfile
    container_name: edge-proxy
    platform: linux/arm64
    cap_add:
      - NET_ADMIN
      - NET_RAW
    environment:
      - PYTHONUNBUFFERED=1
    ports:
      - "8080:8080"
      - "465:465"
    volumes:
      - ./proxy:/home/mitmproxy/scripts
      - ./mitmproxy_certs:/home/mitmproxy/.mitmproxy
    entrypoint: ["mitmdump"]
    command:
      - "-p"
      - "465"
      - "--mode"
      - "reverse:tcp://10.0.1.10:25" # Inoltra tutto il traffico SMTP al mail server i
nterno
      - "--set"
      - "listen_host=0.0.0.0" # Ascolta su tutte le interfacce per accettare traffico
da attaccante e mail server
      - "--set"
      - "confdir=/home/mitmproxy/.mitmproxy" # Usa questa directory per i certificati
e la configurazione persistente
      - "-s"
      - "/home/mitmproxy/scripts/payload_extraction.py" # Script per estrarre payload
e metadata
      - "-s"
      - "/home/mitmproxy/scripts/fingerprinting.py" # Script per fingerprinting JA3
    networks:
      public-net:
          ipv4_address: 10.0.0.5 # IP statico del proxy nella rete pubblica
      private-net:
          ipv4_address: 10.0.1.5 # IP statico del proxy nella rete privata
```

> Il volume `./mitmproxy_certs` garantisce che la **Master CA** non cambi al riavvio del container. Se la CA cambiasse, l'attaccante riceverebbe un errore di validazione del certificato, interrompendo la catena di automazione.

Funzionalità

1. **Terminazione TLS**: Agisce come endpoint per la cifratura dell'attaccante.

   All'avvio, mitmproxy genera `mitmproxy-ca-cert.pem`. Il certificato deve essere installato nel trust-store dell'attaccante (`/usr/local/share/ca-certificates/`). Durante l'handshake, il proxy presenta un certificato per `10.0.0.5` firmato dalla CA fidata, permettendo l'accesso ai dati in chiaro (Plaintext).

   È chiaro che questa risulta essere una semplificazione adottata per l'ambiente di laboratorio, nel caso in cui si volesse adattare la gestione dei flussi TLS dell'Edge Proxy a quelli che sono i workflow reali, avremmo dovuto aumentare l'integrazione con il mail-server. L'Edge Proxy dovrebbe possedere una **CA di firma (Intermediate CA)** fidata dai client aziendali. Quando il client si connette, il proxy genera "al volo" un certificato per il server richiesto.

   In alternativa si sarebbe dovuto installare sull'Edge Proxy un **certificato pubblico valido**, in modo che l'utente esterno possa fidarsi automaticamente.

2. **Threat Intelligence:** Basato su JA3 Fingerprinting. Tramite l'uso della libreria Scapy il sistema esegue un'ispezione passiva dei pacchetti TLS *Client Hello*. L'impronta digitale estratta viene confrontata con database di indicatori di compromissione (**IoC**) forniti da enti come *Abuse.ch*. Questo permette di classificare l'origine della connessione prima della decifratura applicativa, identificando tool automatizzati o agenti malevoli noti (es. *Emotet*) tramite la loro specifica impronta nella negoziazione crittografica.

   Di seguito mostriamo una parte del DB di Abuse.ch:

```
#################################################################
# abuse.ch Suricata JA3 Fingerprint Blacklist (CSV)             #
# For Suricata 4.1.0 or newer                                   #
# Last updated: 2021-08-03 14:33:44 UTC                         #
#                                                               #
# Terms Of Use: https://sslbl.abuse.ch/blacklist/               #
# For questions please contact sslbl [at] abuse.ch              #
#################################################################
#
# ja3_md5,Firstseen,Lastseen,Listingreason
b386946a5a44d1ddcc843bc75336dfce,2017-07-14 18:08:15,2019-07-27 20:42:54,Dridex
8991a387e4cc841740f25d6f5139f92d,2017-07-14 19:02:03,2019-07-28 00:34:38,Adware
cb98a24ee4b9134448ffb5714fd870ac,2017-07-14 19:48:28,2019-05-22 03:22:38,Dridex
1aa7bf8b97e540ca5edd75f7b8384bfa,2017-07-14 20:23:38,2019-07-28 01:38:22,TrickBot
3d89c0dfb1fa44911b8fa7523ef8dedb,2017-07-15 04:23:45,2021-02-01 18:23:25,Adware
bc6c386f480ee97b9d9e52d472b772d8,2017-07-15 10:57:38,2021-03-13 07:33:39,Adware
8f52d1ce303fb4a6515836aec3cc16b1,2017-07-15 19:05:11,2019-07-27 20:00:57,TrickBot
d6f04b5a910115f4b50ecec09d40a1df,2017-07-15 19:42:24,2018-10-14 08:12:51,Dridex
35c0a31c481927f022a3b530255ac080,2017-07-15 19:43:19,2021-04-10 12:54:04,Tofsee
e330bca99c8a5256ae126a55c4c725c5,2017-07-15 19:59:29,2021-01-13 00:29:37,Adware
d551fafc4f40f1dec2bb45980bfa9492,2017-07-15 19:59:29,2020-11-16 13:06:20,Adware
83e04bc58d402f9633983cbf22724b02,2017-07-16 01:32:03,2021-03-02 04:07:43,Adware
b8f81673c0e1d29908346f3bab892b9b,2017-07-16 01:32:03,2021-03-02 04:07:36,Adware
70722097d1fe1d78d8c2164640ab6df4,2017-07-16 02:39:08,2021-05-04 09:52:20,Tofsee
9c2589e1c0e9f533a022c6205f9719e1,2017-07-16 08:37:17,2021-07-25 08:33:18,Adware
849b04bdbd1d2b983f6e8a457e0632a8,2017-07-16 08:37:17,2021-07-25 08:33:18,Adware
```

   Inizialmente, il sistema utilizzava la libreria **Scapy** per il parsing dei byte grezzi all'interno dell'evento `tcp_message`. Tuttavia, l'implementazione finale è passata all'uso delle API native di **mitmproxy 12.x**, sfruttando gli hook specifici del layer TLS (`tls_clienthello`). Questa scelta è stata dettata da criticità strutturali riscontrate con l'approccio basato su Scapy.

Un motivo determinante per l'abbandono di Scapy è stato il **conflitto di esecuzione** tra i vari hook di intercettazione.

L'uso di Scapy all'interno di `tcp_message` costringeva il sistema a competere per lo stesso buffer di memoria utilizzato simultaneamente dallo script di raccolta del payload. In scenari di traffico SMTP asincrono, si verificava spesso una **race condition**: lo script di estrazione del payload (Layer 7) iniziava a processare i comandi SMTP prima che il parser Scapy (Layer 4) avesse terminato l'analisi dell'handshake TLS.

- **Race Conditions**: Spesso lo script di raccolta del payload (Layer 7) "consumava" il buffer prima che Scapy (Layer 4) avesse finito di processare l'handshake.
- **Blocco dell'Evento**: Scapy è sincrono e pesante. Se il parsing del JA3 richiedeva troppo tempo, mitmproxy passava agli eventi successivi (come la lettura del comando SMTP `DATA`) ignorando l'esecuzione del fingerprinting.

L'uso delle API native ha risolto l'instabilità operativa riscontrata con Scapy, dove la natura asincrona del traffico SMTP portava alla perdita del trigger di fingerprinting in favore della sola cattura del payload. L'integrazione nativa assicura che l'identità del client (JA3) sia stabilita prima ancora che il contenuto del messaggio venga processato.

```
import hashlib
import requests
from mitmproxy import tls, tcp
from intelligence import JA3Classifier
import shared_state

# Istanza globale del classificatore
classifier = JA3Classifier()

# Rileva valori GREASE (RFC 8701). Supporta sia interi che altri tipi, filtrando so
lo gli interi validi.
def is_grease(val: int) -> bool:
    if not isinstance(val, int):
        return False
    return (val & 0x0f0f) == 0x0a0a

# Estrae i 5 parametri JA3 in modo rigoroso: Version, Ciphers, Extensions, Elliptic
Curves, EllipticCurveFormats
def get_ja3_fingerprint(ch: tls.ClientHelloData.client_hello) -> str: # ch sarà un
oggetto ClientHelloData.client_hello, che contiene tutte le informazioni necessarie
per estrarre i parametri JA3
    # TLS Version (Valore numerico del record ClientHello)
    # ch.client_version è un bytes di 2 elementi (es. b'\x03\x03')
    try:
        tls_version = str(ch.version)
    except AttributeError:
        # Fallback nel caso in cui la proprietà abbia un nome diverso
        tls_version = str(int.from_bytes(getattr(ch, "client_version", b'\x03\x0
3'), "big"))

    # Cipher Suites (Escludendo GREASE)
    ciphers = "-".join(str(c) for c in ch.cipher_suites if not is_grease(c))

    # Extensions
    # Nelle nuove API, le chiavi del dizionario extensions sono gli ID numerici
    # Dobbiamo estrarre solo gli ID, filtrando i GREASE
    try:
```

```
        raw_exts = ch.extensions.keys() if hasattr(ch.extensions, "keys") else ch.e
xtensions
        ext_ids = [str(e) for e in raw_exts if isinstance(e, int) and not is_grease
(e)]
    except Exception:
        ext_ids = []
    extensions = "-".join(ext_ids)

    # Supported Groups (Parsing manuale dai byte dell'estensione 10)
    supported_groups = ""
    if 10 in ch.extensions:
        raw_groups = ch.extensions[10] # Questi sono i bytes dell'estensione
        groups = []
        try:
            # Il formato è: 2 byte lunghezza totale + N gruppi (2 byte ciascuno)
            # Partiamo dall'indice 2 per saltare la lunghezza
            for i in range(2, len(raw_groups), 2):
                if i + 2 <= len(raw_groups):
                    g = int.from_bytes(raw_groups[i:i+2], "big")
                    if not is_grease(g):
                        groups.append(str(g))
            supported_groups = "-".join(groups)
        except Exception:
            supported_groups = ""

    # EC Point Formats (Parsing manuale dai byte dell'estensione 11)
    ec_formats = ""
    if 11 in ch.extensions:
        raw_formats = ch.extensions[11] # Questi sono i bytes dell'estensione
        try:
            # Il formato è: 1 byte lunghezza lista + N formati (1 byte ciascuno)
            # Partiamo dall'indice 1 per saltare la lunghezza
            formats = [str(raw_formats[i]) for i in range(1, len(raw_formats))]
            ec_formats = "-".join(formats)
        except Exception:
            ec_formats = ""

    # Costruzione stringa JA3 finale
    ja3_string = f"{tls_version},{ciphers},{extensions},{supported_groups},{ec_form
ats}"
    return ja3_string

# Handler per ClientHello, attivato ad ogni handshake TLS
def tls_clienthello(data: tls.ClientHelloData):
    # Estrazione metadati identificativi dal contesto del flusso
    client_ip = data.context.client.peername[0]
    client_id = data.context.client.id
    ch = data.client_hello

    # Generazione della stringa JA3 basata sui parametri TLS (Version, Ciphers, Ext
ensions, Elliptic Curves)
    ja3_string = get_ja3_fingerprint(ch)
    ja3_hash = hashlib.md5(ja3_string.encode()).hexdigest()

    # Interrogazione del database locale (malware_db e scripting_db)
```

```
    category, detail = classifier.classify(ja3_hash)

    # Memorizzazione dello stato nella memoria condivisa per l'analisi successiva d
el Proxy (Nodo 2)
    shared_state.ja3_memory[client_id] = {
        "category": category,
        "detail": detail,
        "hash": ja3_hash
    }

    # Gestione delle due categorie principali: MALWARE vs AUTOMATION/UNKNOWN
    if category == "MALWARE":
        print(f"[JA3-BLOCK] Identificato {detail}. Interruzione immediata di Client
e Server.")

        # Chiudiamo la connessione verso il Mail Server (se già aperta)
        # Questo evita che Postfix rimanga in stato 'connect' nel log senza ricever
e nulla
        data.context.server.error = "Security Policy Block"
        data.context.client.error = "Security Policy Block"

        # Killiamo il flow principale come sicurezza definitiva
        if hasattr(data.context, "flow"):
            data.context.flow.kill()

    else:
        # Se non è malware, il flusso procede verso l'estrazione del payload SMTP
        print(f"[JA3-FINGERPRINT] Classificato come {category}. [JA3:{ja3_hash}] In
oltro al payload extractor.")
```

Come possiamo vedere il seguente codice non fa altro che quanto detto nel paragrafo relativo alla logica di JA3 fingerprinting. Ottenuto l'hash, andiamo poi a realizzare la classificazione tramite una funzione definita nel seguente file:

```
import csv
import os

class JA3Classifier:
    def __init__(self, blacklist_path="/home/mitmproxy/data/ja3_blacklist.csv"):
        self.malware_db = {}

        # Definiamo alcuni hash noti per script Python standard
        self.scripting_db = {
            "3b5074b1b590322741a469f345388c5f": "Python smtplib / requests",
            "ed71105d21b0c098f56b4599f7fb1d74": "Python smtplib (Docker Attacker)",
            "9735870094136e053158434604d55b0b": "Go-http-client"
        }
        self._load_malware_db(blacklist_path)

    # Carica il database di malware da un file CSV, ignorando le righe commentate
(che iniziano con #)
    def _load_malware_db(self, path):
        if os.path.exists(path):
            with open(path, mode='r', encoding='utf-8') as f:
                reader = csv.reader(filter(lambda row: row[0] != '#', f))
```

```
                for row in reader:
                    if len(row) >= 2:
                        self.malware_db[row[0]] = row[1]

    # Classificazione basata su hash JA3
    def classify(self, ja3_hash):

        if ja3_hash in self.malware_db:
            return "MALWARE", self.malware_db[ja3_hash] # Se l'hash è presente nel
database, ritorna MALWARE e il nome della minaccia
        if ja3_hash in self.scripting_db:
            return "AUTOMATION", self.scripting_db[ja3_hash] # Se l'hash è presente
nel database di scripting, ritorna AUTOMATION e il nome dello script
        return "UNKNOWN", "Client non identificato"
```

È bene soffermarci sul fatto che è stata utilizzata una cache logica all'interno del proxy per gestire la logica di decisione in seguito alla classificazione. Infatti notiamo che quando classifichiamo una mail come MALWARE il flusso viene direttamente interrotto (dando completa fiducia ai riferimenti presi come db di hash JA3 associati a MALWARE), mentre quando classifichiamo la mail come AUTOMATION o UNKNOWN, avviene la logica di IPS, descritta al prossimo punto.

3. **Ispezione Semantica (payload_extraction.py)**: Utilizza un intercettore a livello TCP per monitorare la transazione SMTP. Andrà quindi a gestire il contenuto applicativo (Layer 7). Identifica lo stato `DATA` del protocollo e isola il corpo della mail. Preleva il risultato della classificazione dello script precedente, e in base a questa effettua o meno l'inferenza inoltrando il payload al nodo di inferenza.

```
from mitmproxy import tcp
import email # Per analizzare strutture multipart, estrarre i boundary e risolvere
dinamicamente le codifiche di trasferimento (come Base64 e Quoted-Printable).
from email.policy import default
import requests
import shared_state

# Configurazione nodo di inferenza
INFERENCE_URL = "https://10.0.1.20:5000/analyze"
# Percorso del certificato TLS per la comunicazione sicura con il nodo di inferenza
CERT_PATH = "/certs/cert.pem"

session_state = {}

# Funzione per estrarre il testo in chiaro da un payload SMTP, supporta sia messagg
i semplici che multipart, e gestisce dinamicamente le codifiche di trasferimento
def extract_clear_text(raw_bytes: bytes) -> str:
    msg = email.message_from_bytes(raw_bytes, policy=default)
    extracted_text = ""

    # Se il messaggio è multipart, iteriamo su tutte le parti per estrarre il test
o, altrimenti estraiamo direttamente il payload. Gestiamo anche le codifiche di tra
sferimento in modo dinamico, lasciando che la libreria email si occupi di decodific
are correttamente i contenuti.
    if msg.is_multipart():
        for part in msg.walk():
            content_type = part.get_content_type()
            content_disposition = str(part.get("Content-Disposition"))
```

```
            if content_type == "text/plain" and "attachment" not in content_disposi
tion:
                try:
                    extracted_text += part.get_content()
                except Exception:
                    continue
    else:
        if msg.get_content_type() == "text/plain":
            try:
                extracted_text = msg.get_content()
            except Exception:
                pass

    return extracted_text.strip()


def tcp_message(flow: tcp.TCPFlow): # Handler per i messaggi TCP, attivo su ogni pa
cchetto del flusso TCP, ci permette di analizzare i dati in transito e applicare lo
giche IPS basate sui metadati JA3

    # Identificazione immediata
    client_id = flow.client_conn.id

    # Verifichiamo il JA3 prima di fare qualsiasi altra cosa
    ja3_info = shared_state.ja3_memory.get(client_id, {})
    ja3_cat = ja3_info.get("category", "UNKNOWN")

    if ja3_cat == "MALWARE":
        print(f"[IPS-BLOCK] Forzatura chiusura socket per malware JA3: {ja3_info.ge
t('detail')}")

        # Svuotiamo i messaggi pendenti per evitare inoltri accidentali
        if flow.messages:
            flow.messages[-1].content = b""

        # Impostiamo un errore sulla connessione client e server.
        # Questo forza mitmproxy a chiudere i socket TCP immediatamente.
        flow.client_conn.error = "Killed by JA3 Policy"
        if flow.server_conn and flow.server_conn.connected:
            flow.server_conn.error = "Killed by JA3 Policy"

        # Pulizia della memoria per questo client
        if client_id in session_state:
            del session_state[client_id]
        if client_id in shared_state.ja3_memory:
            del shared_state.ja3_memory[client_id]

        # Invece di kill(), usiamo return dopo aver impostato l'errore sulla connes
sione.
        # Mitmproxy rileverà lo stato di errore e abbatterà il flusso senza eccezio
ni.
        return

    # Gestione stato sessione (solo se NON è malware)
    message = flow.messages[-1]
```

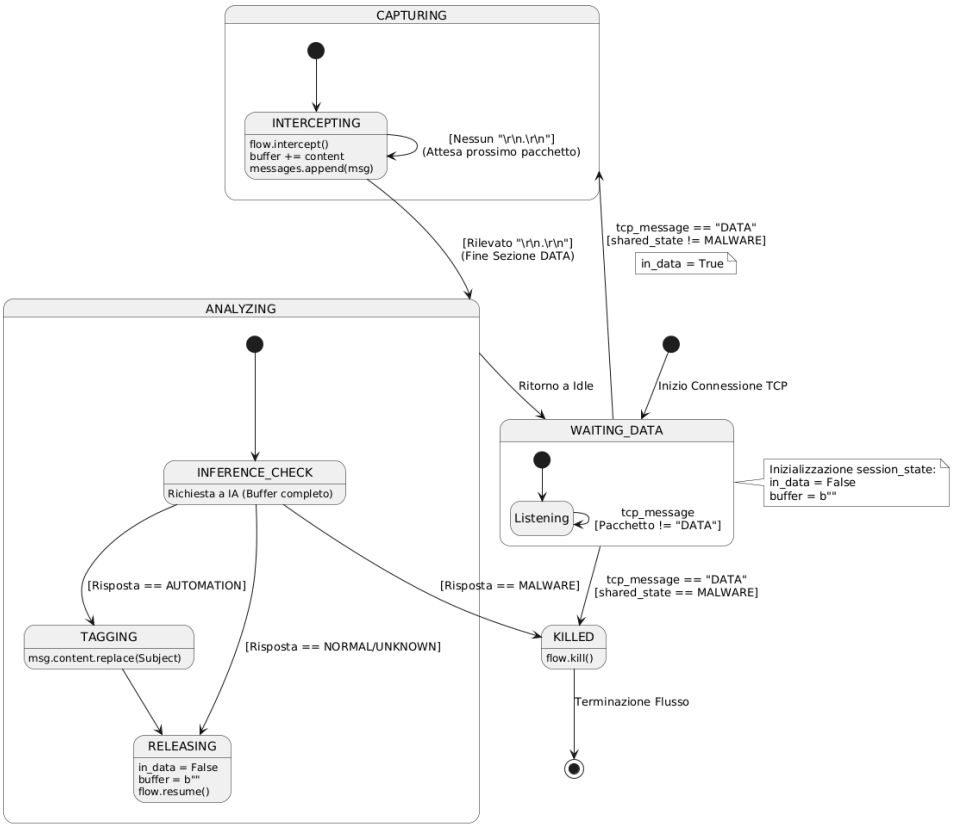
```
    # Inizializziamo lo stato della sessione per questo client se non esiste già, q
uesto ci permette di tracciare se siamo nella sezione DATA di una mail e di accumul
are il payload
    if client_id not in session_state:
        session_state[client_id] = {
            "in_data": False,
            "buffer": b"",
            "intercepted_messages": [],
            "ja3_category": ja3_cat # Salviamo la categoria rilevata
        }


    # Analizziamo solo i messaggi provenienti dal client verso il server, ignorando
quelli di risposta del server, questo perché vogliamo applicare le logiche IPS sui
dati in ingresso
    if message.from_client:
        content_raw = message.content
        state = session_state[client_id]

        # Rilevamento comando DATA
        if b"DATA" == content_raw.upper().strip():

            # LETTURA DALLA MEMORIA CONDIVISA PYTHON USANDO L'ID DEL CLIENT
            ja3_info = shared_state.ja3_memory.get(client_id, {})
            ja3_cat = ja3_info.get("category", "UNKNOWN")
            ja3_detail = ja3_info.get("detail", "Unknown")
            ja3_hash = ja3_info.get("hash", "")

            state["ja3_category"] = ja3_cat
            state["ja3_detail"] = ja3_detail
            state["ja3_hash"] = ja3_hash

            # Se il JA3 è già classificato come MALWARE, killiamo subito
            if ja3_cat == "MALWARE":
                flow.kill()
                return

            # Attiviamo la cattura del payload nella sezione DATA
            state["in_data"] = True
            print(f"[IPS] Comando DATA rilevato. Inizio cattura payload.")
            return

        if state["in_data"]:

            # Intercettazione e Accumulo
            flow.intercept() # Intercettiamo il flusso per analizzare il payload pr
ima di lasciarlo passare
            state["buffer"] += content_raw
            state["intercepted_messages"].append(message)

            # Analisi alla chiusura della mail
            if content_raw.strip() == b"." or content_raw.endswith(b"\r\n.\r\n"): #
Il punto singolo su una riga indica la fine del DATA in SMTP
                print("[IPS] Fine mail. Avvio analisi completa del payload...")
```

```
                # Qui chiamiamo l'IA solo se necessario
                ja3_cat = state.get("ja3_category", "UNKNOWN")
                ja3_detail = state.get("ja3_detail", "Unknown")
                ja3_hash = state.get("ja3_hash", "")

                try:
                    # Estrazione del testo in chiaro tramite il parser MIME
                    clean_text = extract_clear_text(state["buffer"])

                    # Fallback logico: se il parsing fallisce o non trova text/plai
n, passiamo il buffer decodificato
                    if not clean_text:
                        print("[IPS] Attenzione: Nessun testo in chiaro estratto da
l MIME.")
                        clean_text = state["buffer"].decode('utf-8', errors='ignor
e')
                    else:
                        # AGGIUNGI QUESTA RIGA PER VEDERE IL CORPO NEL TERMINALE
                        print(f"\n[IPS-CONTENT-VIEW] --- INIZIO CORPO DECIFRATO ---
\n{clean_text}\n[IPS-CONTENT-VIEW] --- FINE CORPO DECIFRATO ---\n")


                    print(f"[IPS] Invio al nodo di inferenza per la clasificazion
e.\n")

                    # Chiamata al nodo di inferenza
                    response = requests.post(
                        INFERENCE_URL,
                        json={
                            "payload": clean_text,
                            "ja3_classification": ja3_cat,
                            "ja3_detail": ja3_detail,
                            "ja3_hash": ja3_hash
                        },
                        timeout=10,
                        verify=CERT_PATH # Usiamo il certificato TLS per la verific
a della connessione al nodo di inferenza
                    )


                    res = response.json()
                    label = res.get("label", "UNKNOWN") # La label restituita dal
l'IA

                    # Se la classificazione è AUTOMATION, applichiamo un tag specif
ico all'header Subject
                    if label == "AUTOMATION" or label == "MALWARE": # Aggiunto MALW
ARE nel caso in cui l'IA decida di non droppare ma solo taggare
                        ja3_info = state.get("ja3_category", "UNKNOWN")
                        tag = res.get("tag", "[AI-CHECK]")

                        print(f"[IPS-TAGGING] Messaggio classificato come {label} d
al nodo di inferenza. Inserimento tag in corso...{tag}")
```

```
                        # Scorriamo tutti i pacchetti TCP che abbiamo messo in paus
a
                        for msg in state["intercepted_messages"]:
                            if b"Subject: " in msg.content:
                                # Applichiamo la modifica direttamente ai byte del
pacchetto corretto
                                msg.content = msg.content.replace(b"Subject: ", b"S
ubject: " + tag.encode() + b" ")
                                print("[IPS] Header Subject modificato con success
o.")
                                break
                    else:
                        print(f"[IPS] Payload validato. Classificazione: {label}")

                except Exception as e:
                    print(f"[IPS-ERROR] Nodo IA irraggiungibile o errore: {e}")

                    error_tag = b"[AI-UNVERIFIED]"

                    # In caso di errore di comunicazione con il nodo di inferenza,
applichiamo un tag di errore per indicare che il messaggio non è stato analizzato
                    for msg in state["intercepted_messages"]:
                        if b"Subject: " in msg.content:
                            msg.content = msg.content.replace(b"Subject: ", b"Subje
ct: " + error_tag + b" ")
                            break


                state["in_data"] = False
                state["buffer"] = b""
                state["intercepted_messages"] = []


                if client_id in shared_state.ja3_memory:
                    del shared_state.ja3_memory[client_id]

                flow.resume()
```

L'intercettazione del payload SMTP richiede un'ispezione di tipo **Stateful**. Poiché il protocollo SMTP è transazionale, il proxy deve mantenere un automa a stati finiti per identificare la transizione dal piano dei comandi (EHLO, MAIL FROM) al piano dei dati (DATA).

Lo script implementato monitora i segmenti TCP diretti verso l'upstream: al rilevamento di `DATA`, il proxy entra in modalità '**Capture**', isolando il corpo del messaggio fino alla ricezione del delimitatore di fine stream (`\r\n.\r\n`). *Questo garantisce che l'Inference Engine riceva esclusivamente il payload applicativo*, eliminando il rumore generato dal protocollo di trasporto.



Mitmproxy è un proxy applicativo (Layer 7). Quando intercetta un pacchetto TLS, lo "spacchetta" già internamente per gestirlo.

- **Con Scapy**: Prendiamo i byte grezzi (`message.content`) e chiediamo a Scapy di ri-decodificarli. Se il pacchetto TLS è frammentato (diviso in due segmenti TCP), Scapy riceve solo un pezzo e fallisce il parsing, restituendo `None`.
- **Con Mitmproxy Nativo**: Ha già riassemblato i frammenti TCP. Ti dà l'oggetto `client_hello` già pronto, pulito e completo di tutte le estensioni, senza che tu debba preoccuparti degli offset dei byte.

Scapy è una libreria potente ma pesante. In un ambiente di rete ad alte prestazioni (come un Edge Proxy), importare Scapy per leggere solo 5 campi che mitmproxy ha già estratto aggiunge un ritardo.

Nodo inference-engine

Il nodo di inferenza rappresenta il livello cognitivo del sistema **Intrusion Detection System-IDS**.

Operando in un ambiente containerizzato e isolato all'interno della rete privata (`private-net`), espone un'API RESTful asincrona che riceve i payload decifrati dall'Edge Proxy. L'obiettivo primario del nodo è discriminare tra traffico generato da operatori umani e contenuti sintetizzati da Modelli Linguistici di Grandi Dimensioni (LLM), mitigando tentativi di phishing automatizzato e ingegneria sociale.

```
inference-engine:
    build: ./analysis  # Useremo un Dockerfile per installare le librerie AI
    container_name: inference-engine
    platform: linux/arm64
    networks:
      private-net: # Totalmente isolato dall'attaccante
        ipv4_address: 10.0.1.20 # IP statico del nodo di inferenza
    # Questo nodo non ha porte esposte all'esterno: è accessibile solo dal proxy
    volumes:
      - shared-certs:/certs
      - ./models:/app/models # Questo volume conterrà il file dei pesi precedentemente
```

```
calcolati per il modello
    # Comando che genera il certificato TLS al volo e poi avvia Uvicorn in HTTPS
    command: >
      sh -c "openssl req -x509 -newkey rsa:4096 -nodes -out /certs/cert.pem -keyout /c
erts/key.pem -days 365 -subj '/CN=inference-engine' -addext 'subjectAltName=IP:10.0.1.
20,DNS:inference-engine'
      && uvicorn main:app --host 0.0.0.0 --port 5000 --ssl-keyfile /certs/key.pem --ss
l-certfile /certs/cert.pem"
```

Il nodo ha un funzionamento espresso completamente nel file `main.py`. Questo opera come un microservizio **FastAPI** isolato nella rete privata (`10.0.1.20`), il nodo riceve payload testuali crittografati tramite TLS e restituisce una classificazione multidimensionale basata sulla fusione di metriche statistiche e modelli neurali.

Per ottimizzare le prestazioni su hardware con risorse limitate, il sistema adottato per il nodo di inferenza prevede l'uso di un Transformer encoder-only (`roberta-base`).

L'addestramento è stato eseguito in ambiente **Google Colab** sfruttando GPU T4 per gestire il dataset quadri-classe (Human-Legit, Human-Phish, LLM-Legit, LLM-Phish).

https://www.kaggle.com/datasets/francescogreco97/human-llm-generated-phishing-legitimate-emails?resource=download

Questo addestramento si è concentrato solo sugli adapter del modello. I pesi ottenuti sono poi stati applicati all'avvio del programma tramite la libreria `PEFT`.

Come anticipato nella sezione relativa all'Inferenza AI, non ci affidiamo a un singolo verdetto, ma esegue una validazione incrociata tra due sensori indipendenti:

Utilizzando un modello `GPT-2`, il sistema calcola la **Perplexity (PP)** del testo.

- **Valori Bassi**: Indicano una struttura linguistica altamente prevedibile, tipica dei testi generati da Large Language Models (LLM).
- **Soglia di Override**: Se $PP < 70$, il sistema attiva un override statistico che classifica il contenuto come **AUTOMATION**, indipendentemente dall'intuizione neurale, per mitigare il bias di RoBERTa verso il linguaggio formale.

Il classificatore neurale analizza l'**intento** del messaggio:

- Identifica i trigger tipici del phishing (urgenza, richieste di credenziali, minacce).
- Assegna una delle 4 classi addestrate, che viene poi validata dal filtro di Perplexity per determinare la label finale (`NORMAL`, `SUSPICIOUS`, o `AUTOMATION`).

Flusso di Esecuzione

1. **Ricezione**: Il server FastAPI accetta una richiesta POST in HTTPS.
2. **Pre-processing**: Filtro sulla lunghezza minima (150 caratteri) per evitare falsi positivi su testi troppo brevi.
3. **Calcolo Parallelo**: Esecuzione simultanea del calcolo della Perplexity e dell'inferenza RoBERTa.
4. **Integrazione JA3**: Se i sensori testuali sono incerti, il sistema utilizza il fingerprinting JA3 come ultima rete di sicurezza per identificare bot noti.
5. **Risposta**: Restituzione di un JSON contenente la `label` (azione dell'IPS) e un `tag` descrittivo per la modifica degli header email.

Nodo mail-server

L'obiettivo è la creazione di un **Mail Transfer Agent (MTA)** interno, operante in una rete segregata (`private-net`), che accetti comunicazioni SMTP esclusivamente dall'Edge Proxy. Per massimizzare l'efficienza dell'ispezione, la comunicazione interna avviene in **cleartext** (senza TLS), delegando la sicurezza del trasporto esclusivamente al proxy perimetrale.

Il nodo è basato sull'immagine `boky/postfix`, scelta per la sua leggerezza e compatibilità con architetture ARM64.

Componenti Chiave

- **Supervisord**: Gestore dei processi che mantiene attivi contemporaneamente il demone Postfix e il servizio di logging `rsyslog`.
- **LMDB (Lightning Memory-Mapped Database)**: Formato ad alte prestazioni utilizzato per le tabelle di lookup (alias e mappe).

```
mail-server:
    # Postfix è leggero, ma usiamo un'immagine compatibile ARM
    build: ./mail # Useremo un Dockerfile per configurare Postfix in modo semplice
    container_name: mail-server
    platform: linux/arm64
    environment:
      # Definisce l'identità del server (FQDN)
      - POSTFIX_myhostname=mail.netsentinel.local
      - POSTFIX_mydestination=mail.netsentinel.local,netsentinel.local,localhost.local
domain,localhost # Definisce i domini per cui il server accetterà mail (il nostro domi
nio simulato)
      - POSTFIX_mynetworks=10.0.1.0/24,127.0.0.0/8 # Accettiamo mail solo dalla nostra
rete privata e localhost
      - ALLOWED_SENDER_DOMAINS=netsentinel.local,attacker.local,localhost # Whitelist:
accetta mail da questi domini simulati
      - LOG_LEVEL=debug # Log più dettagliati per il debug
      # Permettiamo TLS ma non lo richiediamo (STARTTLS è opzionale), nelle fasi succe
ssive potremmo aumentare la sicurezza
      - POSTFIX_smtpd_recipient_restrictions=permit_mynetworks,permit_sasl_authenticat
ed,reject_unauth_destination
      - POSTFIX_smtpd_tls_security_level=may
      - POSTFIX_smtpd_tls_auth_only=no
    networks:
      private-net: # Totalmente isolato dall'attaccante
        ipv4_address: 10.0.1.10 # IP statico per il server
```

Nodo attacker-node

Questo nodo è progettato per simulare un client ostile o un bot automatizzato che tenta di veicolare contenuti generati da IA (phishing sintetico) all'interno dell'organizzazione.

Funzionalità Principali

1. **Script di Attacco (`attack_sim.py`)**: Uno script Python che utilizza la libreria `smtplib` per connettersi al proxy. È configurato per inviare mail con diverse varianti di testo (naturale vs sintetico) per testare la soglia di sensibilità del sistema.

   ```
   # Questo script simula un attacco di phishing inviando un'email dannosa a una vitti
   ma attraverso un server proxy.
   # L'email contiene un messaggio di phishing che cerca di indurre la vittima a clicc
   are su un link e fornire informazioni sensibili.
   # La mail viene inviata utilizzando il protocollo MIME e codificata in base64.
   import smtplib
   from email.mime.multipart import MIMEMultipart
   from email.mime.text import MIMEText
   from email.encoders import encode_base64
   import ssl

   PROXY_HOST = "10.0.0.5"
   PROXY_PORT = 465
   ```

```
SENDER = "attacker@attacker.local"
RECIPIENT = "victim@netsentinel.local"

msg = MIMEMultipart()
msg['Subject'] = 'Email di prova'
msg['From'] = SENDER
msg['To'] = RECIPIENT

testo_phishing = """HURGENT WARNING!!! We have detected a serious error on your ban
k profile.
                                        Please you must update your details immedia
tely to avoid losing access to your funds. Click here: http://secure-fast-web.it/lo
gin and enter your password and the code you receive via SMS.
                                        Do not waste time, you only have 10 minutes
before the definitive closure of your bank account.
                                        Thanks security office."""

parte_testo = MIMEText(testo_phishing, 'plain', 'utf-8')

encode_base64(parte_testo)

msg.attach(parte_testo)

# In questo caso, utilizziamo SMTP_SSL per stabilire una connessione sicura con il
server proxy sulla porta 465.
try:
    context = ssl.create_default_context()

    server = smtplib.SMTP_SSL(PROXY_HOST, PROXY_PORT, context=context)
    server.set_debuglevel(1)

    server.send_message(msg)
    server.quit()

except Exception as e:
    print(f"\n[ERRORE] Impossibile inviare l'email: {e}")
```

2. **Vettore di Iniezione**: Rappresenta il punto di origine dei metadati (come il fingerprint JA3) che verranno utilizzati per il profiling del client nelle fasi successive del progetto.

Gestione della crittografia

Per permettere al proxy di ispezionare il contenuto della mail, abbiamo implementato la **TLS Termination**. Questo significa che la sessione cifrata non avviene tra Attaccante e Mail Server, ma si ferma sul Proxy.

- **Segmento A (Attaccante ↔ Proxy)**: Connessione cifrata (SMTPS).
- **Segmento B (Proxy ↔ Mail Server)**: Connessione in chiaro (SMTP).

Il proxy deve "presentarsi" all'attaccante come se fosse il destinatario legittimo. Per fare ciò, sono necessari due elementi:

- **Certificato del Server**: Il proxy deve possedere un certificato che dichiari di essere `mail.netsentinel.local`.
- **Chiave Privata**: Necessaria per avviare l'handshake TLS e decifrare i dati inviati dall'attaccante.

Come detto nel paragrafo relativo a **Mitmproxy**, al primo avvio viene generata una coppia di chiavi e un certificato Root CA.

In particolare, `mitmproxy-ca.pem`: È il file più critico. Contiene sia il **certificato Root CA** (pubblico) che la **chiave privata** (privata).

Abbiamo configurato il proxy per utilizzare un certificato (auto-firmato in contesti di laboratorio) che copre il nome di dominio del server mail. Senza questo certificato, l'attaccante non riuscirebbe nemmeno ad avviare la connessione (*SSL Handshake Failed*).

In un attacco reale o in una simulazione, il client (l'attaccante) deve "fidarsi" del certificato presentato dal proxy.

- **Il Problema**: Se il certificato del proxy è auto-firmato, il client lo rifiuterà perché non è emesso da una Certification Authority (CA) nota.
- **La Soluzione**: È necessario istruire l'attaccante affinché:
  1. Accetti certificati non verificati (opzione meno sicura).
  2. O, preferibilmente, si carica il certificato CA del proxy tra i suoi "Trusted Roots".

Nel nostro caso, la catena di fiducia è stata stabilita importando il certificato *Root CA* di mitmproxy all'interno del trust store dell'attaccante.

```
 volumes:
      - ./attacker:/scripts #
      - ./mitmproxy_certs:/mnt/cert_source:ro # Montiamo qui i certificati originali,
cioè copio i certificati di mitmproxy in una cartella temporanea all'interno del conta
iner dell'attaccante
    # Nelle prossime righe, copiamo il certificato CA di mitmproxy nella posizione cor
retta all'interno del container dell'attaccante e aggiorniamo le CA del sistema, in mo
do che l'attaccante possa comunicare senza errori SSL con il proxy
    command: >
      sh -c "cp /mnt/cert_source/mitmproxy-ca-cert.pem /usr/local/share/ca-certificate
s/mitmproxy-ca.crt &&
             update-ca-certificates &&
             tail -f /dev/null"
```

Questa operazione abilita l'ispezione del traffico TLS senza innescare alert di sicurezza lato client. È la dimostrazione pratica di come un'organizzazione possa monitorare il traffico cifrato in uscita o in entrata, a patto di avere il controllo sui certificati di root dei dispositivi endpoint.

In produzione, l'impiego di una CA privata come quella usata nel laboratorio sarebbe sostituito da certificati emessi da entità terze riconosciute (**Public CAs**). Tuttavia, la logica di **TLS Termination** rimarrebbe identica: il Proxy agisce come endpoint crittografico, centralizzando la gestione della sicurezza e permettendo una visibilità granulare sul traffico applicativo prima della consegna finale al server di posta.

Oltre alla comunicazione con gli endpoint esterni alla rete privata andiamo a gestire la messa in sicurezza dell'intero canale anche tra il **Proxy** e il **nodo di inferenza**.

Questo si ottiene implementando **TLS (Transport Layer Security)** direttamente sul server di inferenza, trasformando la comunicazione da HTTP a HTTPS.

Anche in questo caso utilizzeremo un certificato auto-firmato. Per farlo useremo **OpenSSL.**

Per garantire che l'ambiente sia conforme agli standard di produzione dobbiamo generare dinamicamente la **chiave RSA** e il **certificato X.509**, li andremo poi a salvare in un volume condiviso in modo che l'Edge Proxy possa leggerlo in tempo reale, ed essere quindi configurato adeguatamente.

```
# Comando che genera il certificato TLS al volo e poi avvia Uvicorn in HTTPS
command: >
      sh -c "openssl req -x509 -nodes -days 365 -newkey rsa:2048 -keyout /certs/key.pe
m -out /certs/cert.pem -subj '/CN=10.0.1.20' &&
```

```
              uvicorn main:app --host 0.0.0.0 --port 5000 --ssl-keyfile /certs/key.pem
--ssl-certfile /certs/cert.pem"
```

Test

Caso completo di test del Laboratorio

In questa sezione andremo ad analizzare come si comporta il laboratorio realizzato.

Innanzitutto, sarà necessario buildare i container su Docker, operazione semplicissima che verrà avviata dal semplice comando `docker-compose build`, prima di usare il comando sarà necessario andare mettersi nel percorso `LLM_ENCRYPTED_FLOW_DETECTION/Lab/`.

In questo momento tutti i container sono in fase di costruzione per quanto definito nel `docker-compose.yml` e nei reciproci `Dockerfile`.

In particolare, verrano eseguite le seguenti operazioni:

- Installazione di Mitmproxy[**edge-proxy**].
- Download Modelli[**inference-engine**].
- Compilazione Postfix[**mail-server**].
- Setup attaccante[**attacker-node**].

Fatto ciò andremo allora ad avviare i container con il comando `docker-compone up`, tralasciamo l'uso dello switch `-d` per poter visualizzare i log dei nodi.

L'avvio restituirà quanto segue sul terminale.

```
luiicare@192 Lab % docker-compose up
[+] Running 6/6
 ✔ Network lab_public-net      Created                                    0.1s
 ✔ Network lab_private-net     Created                                    0.1s
 ✔ Container inference-engine  Created                                    0.1s
 ✔ Container mail-server       Created                                    0.3s
 ✔ Container attacker-node     Created                                    0.1s
 ✔ Container edge-proxy        Created                                    0.1s
Attaching to attacker-node, edge-proxy, inference-engine, mail-server
```

Vediamo nel dettaglio cosa accade in ogni singolo container.

mail-server

```
luiicare@192 Lab % docker logs -f mail-server
***** POSTFIX STARTING UP (debian) *****
▸ NOTE  System accounts: postfix=100:102, opendkim=101:104. Careful when switching distros.
▸ INFO  Not setting any timezone for the container
▸ DEBUG /tmp writable.
▸ INFO  Using plain log format for rsyslog.
▸ INFO  Removing /var/log/mail.log from /etc/logrotate.d/rsyslog
▸ NOTE  Emails in the logs will not be anonymized. Set ANONYMIZE_EMAILS to enable this feature.
▸ DEBUG Reowning root: /var/spool/postfix/
▸ DEBUG Reowning root: /var/spool/postfix/pid/
▸ DEBUG Reowning postfix:postdrop /var/spool/postfix/private/
▸ DEBUG Reowning postfix:postdrop /var/spool/postfix/public/
▸ INFO  Preparing files for Postfix chroot:
cp: '/etc/localtime' and '/var/spool/postfix/etc/localtime' are the same file
        mkdir: created directory '/var/spool/postfix/usr'
        mkdir: created directory '/var/spool/postfix/usr/share'
        mkdir: created directory '/var/spool/postfix/usr/share/zoneinfo'
        Copying /usr/share/zoneinfo -> /var/spool/postfix/usr/share/zoneinfo
        '/etc/localtime' -> '/var/spool/postfix/etc/localtime'
        '/etc/nsswitch.conf' -> '/var/spool/postfix/etc/nsswitch.conf'
        '/etc/resolv.conf' -> '/var/spool/postfix/etc/resolv.conf'
        '/etc/services' -> '/var/spool/postfix/etc/services'
        '/etc/host.conf' -> '/var/spool/postfix/etc/host.conf'
        '/etc/hosts' -> '/var/spool/postfix/etc/hosts'
        '/etc/passwd' -> '/var/spool/postfix/etc/passwd'
▸ NOTE  Switching default_database_type to lmdb to ensure cross-distro compatibility.
▸ WARN  Detected old hash: and btree: references in the config file, which are not supported anymore. Upgrading to lmdb:
▸ DEBUG Creating new postalias for lmdb:/etc/aliases.
▸ DEBUG Creating new postalias for lmdb:/etc/aliases.
▸ INFO  Using unlimited message size.
▸ INFO  Setting smtp_tls_security_level: may
▸ NOTE  Postfix is configured to deliver messages directly (without relaying). Make sure your DNS is setup properly! If unsure, read the docs.
▸ NOTE  Using custom allowed networks: 10.0.1.0/24,127.0.0.0/8
▸ INFO  Debugging is disabled.
▸ INFO  Setting up allowed SENDER domains: netsentinel.local,attacker.local,localhost
▸ DEBUG DKIM_AUTOGENERATE not set -- you will need to provide your own keys.
▸ INFO  No DKIM keys found, will not use DKIM.
▸ INFO  Applying custom postfix setting: message_size_limit=0
▸ INFO  Applying custom postfix setting: mydestination=mail.netsentinel.local,netsentinel.local,localhost.localdomain,localhost
▸ INFO  Applying custom postfix setting: myhostname=mail.netsentinel.local
▸ INFO  Applying custom postfix setting: mynetworks=10.0.1.0/24,127.0.0.0/8
▸ INFO  Applying custom postfix setting: smtp_tls_security_level=may
▸ INFO  Applying custom postfix setting: smtpd_recipient_restrictions=permit_mynetworks,permit_sasl_authenticated,reject_unauth_destination
▸ INFO  Applying custom postfix setting: smtpd_tls_auth_only=no
▸ INFO  Applying custom postfix setting: smtpd_tls_security_level=may
▸ NOTE  Starting: rsyslog, crond, postfix
2026-03-05 14:42:50,753 INFO Set uid to user 0 succeeded
2026-03-05 14:42:50,755 INFO supervisord started with pid 8
2026-03-05 14:42:51,761 INFO spawned: 'cron' with pid 441
2026-03-05 14:42:51,764 INFO spawned: 'opendkim' with pid 442
2026-03-05 14:42:51,767 INFO spawned: 'postfix' with pid 443
2026-03-05 14:42:51,771 INFO spawned: 'rsyslog' with pid 445
2026-03-05T14:42:51.783062+00:00 INFO    rsyslogd: [origin software="rsyslogd" swVersion="8.2504.0" x-pid="445" x-info="https://www.rsyslog.com"] start
2026-03-05 14:42:51,783 INFO success: cron entered RUNNING state, process has stayed up for > than 0 seconds (startsecs)
2026-03-05 14:42:51,783 INFO success: postfix entered RUNNING state, process has stayed up for > than 0 seconds (startsecs)
2026-03-05T14:42:52.823488+00:00 INFO    postfix/postfix-script[992]: starting the Postfix mail system
2026-03-05T14:42:52.829761+00:00 INFO    postfix/master[993]: daemon started -- version 3.10.5, configuration /etc/postfix
2026-03-05 14:42:53,834 INFO success: rsyslog entered RUNNING state, process has stayed up for > than 2 seconds (startsecs)
2026-03-05 14:42:56,843 INFO success: opendkim entered RUNNING state, process has stayed up for > than 5 seconds (startsecs)
```

Possiamo osservare come Postfix crei una gerarchia di directory isolata dove vengono copiati i file di sistema strettamente necessari. Oltre a eseguire una manutenzione automatica delle tabelle di lookup passando al formato moderno LMDB (consentendo accessi simultanei estremamente veloci alle tabelle degli indirizzi).

La parte più interessante è quella relativa alla configurazione della Policy di rete, in quando viene riconosciuto il dominio `netsentinel.local` come destinazione locale, viene limitato l'accesso esclusivamente alla sottorete `10.0.1.0/24`, accettando solo mail provenienti dalla sotto rete, in questa fase contenente solo l'**edge-proxy.** Inoltre viene impostato `smtpd_tls_security_level=may` consentendo la ricezione di traffico cifrato dal proxy.

edge-proxy

```
luiicare@192 Lab % docker logs -f edge-proxy
[14:42:46.100] Loading script /home/mitmproxy/scripts/payload_extraction.py
[14:42:46.192] Loading script /home/mitmproxy/scripts/fingerprinting.py
[14:42:46.195] reverse proxy to tcp://10.0.1.10:25 listening at *:465.
```

Il proxy carica gli script necessari per effettuare il fingerprint JA3 e gestire la mail al livello applicativo, gestendo la logica di comunicazione con il nodo di inferenza, ed eventualmente, modificando l'oggetto della mail.

Infine si mette in ascolto sulla porta pubblica 465, operando come reverse proxy (con protocollo TCP) sulla porta 25 del **mail-server.**

inference-engine

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
[AI NODE] Inizializzazione modelli in corso...
Loading weights: 100%|          | 148/148 [00:00<00:00, 2012.47it/s, Materializing param=transformer.wte.weight]
Loading weights: 100%|          | 197/197 [00:00<00:00, 2095.57it/s, Materializing param=roberta.encoder.layer.11.output.dense.weight]
RobertaForSequenceClassification LOAD REPORT from: roberta-base
Key                             | Status     |
--------------------------------+------------+-
lm_head.layer_norm.weight       | UNEXPECTED |
lm_head.bias                    | UNEXPECTED |
roberta.embeddings.position_ids | UNEXPECTED |
lm_head.dense.weight            | UNEXPECTED |
lm_head.dense.bias              | UNEXPECTED |
lm_head.layer_norm.bias         | UNEXPECTED |
classifier.out_proj.bias        | MISSING    |
classifier.dense.weight         | MISSING    |
classifier.out_proj.weight      | MISSING    |
classifier.dense.bias           | MISSING    |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING       :those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.
INFO:     Started server process [7]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on https://0.0.0.0:5000 (Press CTRL+C to quit)
```

Al momento dell'avvio, il container contatta l'**Hugging Face Hub** per scaricare i parametri pre-addestrati del modello `roberta-base`.

Fatto questo si procede con il caricamento dei pesi:

- Il log riporta un "LOAD REPORT", che spiega come RoBERTa venga adattato al compito di rilevamento phishing.
- Il log mostra il caricamento simultaneo di un secondo set di pesi (148/148), riferibili al modello **GPT-2**.

Con questo possiamo ritenere conclusa la fase di inizializzazione.

Possiamo allora procedere a testare il comportamento del Proxy in relazione ad una serie di mail inviate dall'attaccante.

Mail 1

```
# Questo script simula un attacco di phishing inviando un'email dannosa a una vittima
attraverso un server proxy.
# L'email contiene un messaggio di phishing che cerca di indurre la vittima a cliccare
su un link e fornire informazioni sensibili.
# La mail viene inviata utilizzando il protocollo MIME e codificata in base64.
import smtplib
from email.mime.multipart import MIMEMultipart
from email.mime.text import MIMEText
from email.encoders import encode_base64
import ssl

PROXY_HOST = "10.0.0.5"
PROXY_PORT = 465

SENDER = "attacker@attacker.local"
RECIPIENT = "victim@netsentinel.local"

msg = MIMEMultipart()
msg['Subject'] = 'Email di prova'
msg['From'] = SENDER
msg['To'] = RECIPIENT

testo_phishing = """HURGENT WARNING!!! We have detected a serious error on your bank p
rofile.
                                        Please you must update your details immediatel
y to avoid losing access to your funds.
                                        Click here: http://secure-fast-web.it/login an
d enter your password and the code you receive via SMS.
                                        Do not waste time, you only have 10 minutes be
fore the definitive closure of your bank account.
                                        Thanks security office."""

parte_testo = MIMEText(testo_phishing, 'plain', 'utf-8')

encode_base64(parte_testo)

msg.attach(parte_testo)

# In questo caso, utilizziamo SMTP_SSL per stabilire una connessione sicura con il ser
ver proxy sulla porta 465.
try:
```

```
    context = ssl.create_default_context()

    server = smtplib.SMTP_SSL(PROXY_HOST, PROXY_PORT, context=context)
    server.set_debuglevel(1)

    server.send_message(msg)
    server.quit()

except Exception as e:
    print(f"\n[ERRORE] Impossibile inviare l'email: {e}")
```

L'attaccante invia il payload tramite **SMTPS (Implicit TLS)** sulla porta 465. Poiché il testo del phishing è cifrato all'interno del tunnel TLS e ulteriormente codificato in **Base64**, stiamo testando se l'**Edge-Proxy** è configurato correttamente per:

- **Terminare il tunnel TLS**:.
- **Parsing MIME**.

Possiamo notare come il corpo della mail abbia tutte le caratteristiche note delle mail di phishing.

Lo scopo è vedere se il l'edge-proxy è in grado di riconoscere queste caratteristiche.

Possiamo visualizzare al terminale l'invio della mail.

1. **Handshake e Negoziazione (fase EHLO)**.

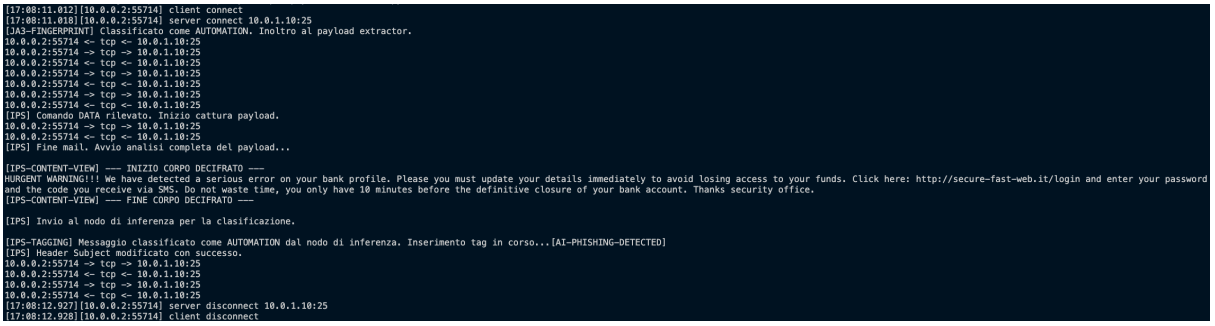
   In cui il sistema risponde identificandosi e elencando le funzionalità supportate.

2. **Costruzione della 'busta'.**

   Possiamo visualizzare i messaggi di invio di `FROM`, `TO`.

3. **Ricezione della conferma.**

All'invio, il proxy intercetterà la mail, effettuando da prima un'analisi di fingerprinting statica secondo la logica di JA3, per poi inviare il risultato con il corpo della mail al nodo di inferenza.



Visualizziamo ora il comportamento del nodo di inferenza.

```
[AI NODE] Metriche - PP: 43.25 | Classe Originale: 1 | Label Finale: AUTOMATION | Tag: [AI-PHISHING-DETECTED]
INFO:     10.0.1.5:35098 - "POST /analyze HTTP/1.1" 200 OK
```

Banalmente abbiamo stampato quelli che sono i valori di Perplexity e la classificazione effettuata dal modello di classificazione.

```
2026-03-05T17:08:11.051567+00:00 INFO    postfix/smtpd[2759]: connect from edge-proxy.lab_private-net[10.0.1.5]
2026-03-05T17:08:11.079461+00:00 INFO    postfix/smtpd[2759]: 1359136F507: client=edge-proxy.lab_private-net[10.0.1.5]
2026-03-05T17:08:12.914863+00:00 INFO    postfix/cleanup[2762]: 1359136F507: message-id=<>
2026-03-05T17:08:12.918450+00:00 INFO    postfix/qmgr[949]: 1359136F507: from=<attacker@attacker.local>, size=1153, nrcpt=1 (queue active)
2026-03-05T17:08:12.921166+00:00 INFO    postfix/smtpd[2759]: disconnect from edge-proxy.lab_private-net[10.0.1.5] ehlo=1 mail=1 rcpt=1 data=1 quit=1 commands=5
2026-03-05T17:08:12.933877+00:00 INFO    postfix/local[2763]: 1359136F507: to=<root@mail.netsentinel.local>, orig_to=<victim@netsentinel.local>, relay=local, delay=1.9, delays=1.9/0.01/0/0.01, dsn=2.0.0, status=sent (delivered t
o mailbox)
2026-03-05T17:08:12.934234+00:00 INFO    postfix/qmgr[949]: 1359136F507: removed
```

A questo punto possiamo controllare se il mail server ha effettivamente ricevuto la mail con la modifica relativa alla detection, eseguendo il comando `docker exec -it mail-server cat /var/mail/root`, possiamo visualizzare la modifica e la mail direttamente all'interno della mailbox del *root*, in quanto non specificato esplicitamente l'utente *victim.*

```
From attacker@attacker.local  Thu Mar  5 17:08:12 2026
Return-Path: <attacker@attacker.local>
X-Original-To: victim@netsentinel.local
Delivered-To: victim@netsentinel.local
Received: from [10.0.0.2] (edge-proxy.lab_private-net [10.0.1.5])
        by mail.netsentinel.local (Postfix) with ESMTP id 1359136F507
        for <victim@netsentinel.local>; Thu, 05 Mar 2026 17:08:11 +0000 (UTC)
Content-Type: multipart/mixed; boundary="===============2849812625149663781==="
MIME-Version: 1.0
Subject: [AI-PHISHING-DETECTED] Email di prova
From: attacker@attacker.local
To: victim@netsentinel.local

--===============2849812625149663781==
Content-Type: text/plain; charset="utf-8"
MIME-Version: 1.0
Content-Transfer-Encoding: base64
Content-Transfer-Encoding: base64

SFVSR0VOVCBXQVJOSU5HISEhIFdlIGhhdmUgZGV0ZWN0ZWQgYSBzZXJpb3VzIGVycm9yIG9uIHlv
dXIgYmFuayBwcm9maWxlLiBQbGVhc2UgeW91IG11c3QgdXBkYXRlIHlvdXIgZGV0YWlscyBpbW1l
ZGlhdGVseSB0byBhdm9pZCBsb3NpbmcgYWNjZXNzIHRvIHlvdXIgZnVuZHMuIENsaWNrIGhlcmU6
IGh0dHA6Ly9zZWN1cmUtZmFzdC13ZWIuaXQvbG9naW4gYW5kIGVudGVyIHlvdXIgcGFzc3dvcmQg
YW5kIHRoZSBjb2RlIHlvdSByZWNlaXZlIHZpYSBTTVMuIERvIG5vdCB3YXN0ZSB0aW1lLCB5b3Ug
b25seSBoYXZlIDEwIG1pbnV0ZXMgYmVmb3JlIHRoZSBkZWZpbml0aXZlIGNsb3N1cmUgb2YgeW91
ciBiYW5rIGFjY291bnQuIFRoYW5rcyBzZWN1cml0eSBvZmZpY2Uu

--===============2849812625149663781==--
```

Rilevazione della categoria AUTOMATION con JA3

Per testare la rilevazione della categoria **AUTOMATION**, dobbiamo creare uno script che utilizzi una libreria TLS standard di Python (come `smtplib` con `ssl`), per farlo, abbiamo inserito all'interno del file `intelligence.py` un database contenente gli hash JA3.

```
# Definiamo alcuni hash noti per script Python standard
        self.scripting_db = {
            "3b5074b1b590322741a469f345388c5f": "Python smtplib / requests",
            "ed71105d21b0c098f56b4599f7fb1d74": "Python smtplib (Docker Attacker)",
            "9735870094136e053158434604d55b0b": "Go-http-client"
        }
```

È stato inoltre scritto lo script menzionato sopra, che riportiamo in seguito per completezza:

```
import smtplib
import ssl
```

```
from email.mime.text import MIMEText
from email.mime.multipart import MIMEMultipart

def send_automation_mail():
    # Puntiamo al file originale nel volume montato, che è un PEM valido
    # Il percorso deve essere quello definito nel docker-compose per 'mnt/cert_source'
    CERT_FILE = "/mnt/cert_source/mitmproxy-ca-cert.pem"

    # Creiamo un contesto SSL ultra-specifico
    context = ssl.SSLContext(ssl.PROTOCOL_TLS_CLIENT)

    # Carichiamo SOLO questo certificato come autorità fidata
    try:
        context.load_verify_locations(cafile=CERT_FILE)
    except Exception as e:
        print(f"Impossibile leggere il file certificato: {e}")
        return

    context.check_hostname = False

    msg = MIMEMultipart()
    msg['From'] = "attacker@attacker.local"
    msg['To'] = "victim@netsentinel.local"
    msg['Subject'] = "Test JA3"
    msg.attach(MIMEText("Simulazione attacco", 'plain'))

    try:
        print(f"Tentativo di connessione con certificato: {CERT_FILE}")
        with smtplib.SMTP_SSL("10.0.0.5", 465, context=context) as server:
            server.sendmail("attacker@attacker.local", "victim@netsentinel.local", ms
g.as_string())
            print("Mail inviata con successo!")
    except Exception as e:
        print(f"[ERROR] Fallimento SSL/TLS: {e}")

if __name__ == "__main__":
    send_automation_mail()
```

Dopo aver attivato l'ambiente docker, andiamo allora ad eseguire lo script sul nodo **attacker-node:**

```
luiicare@192 NetSentinel % docker exec -it attacker-node bash
root@37eec0135f08:/scripts# python3 /scripts/a
attack_sim.py       automation_test.py
root@37eec0135f08:/scripts# python3 /scripts/automation_test.py
Tentativo di connessione con certificato: /mnt/cert_source/mitmproxy-ca-cert.pem
Mail inviata con successo!
root@37eec0135f08:/scripts#
```

Possiamo osservare dei log sul nodo **edge-proxy**:

```
luiicare@192 NetSentinel % docker logs edge-proxy
[14:07:36.133] Loading script /home/mitmproxy/scripts/payload_extraction.py
[14:07:36.136] Loading script /home/mitmproxy/scripts/fingerprinting.py
[14:07:36.190] reverse proxy to tcp://10.0.1.10:25 listening at *:465.
[14:08:09.828][10.0.0.2:40940] client connect
[14:08:09.837][10.0.0.2:40940] server connect 10.0.1.10:25
------------------------------
[JA3 ENGINE] Source IP: 10.0.0.2
[JA3 ENGINE] RAW String: 771,4866-4867-4865-49196-49200-49195-49199-52393-52392-49188-49192-49187-49191-159-158-107-103,,,
[JA3 ENGINE] MD5 Hash: ed71105d21b0c098f56b4599f7fb1d74
[ACTION] Forwarding to Inference: AUTOMATION
10.0.0.2:40940 <- tcp <- 10.0.1.10:25
10.0.0.2:40940 -> tcp -> 10.0.1.10:25
10.0.0.2:40940 <- tcp <- 10.0.1.10:25
10.0.0.2:40940 -> tcp -> 10.0.1.10:25
10.0.0.2:40940 <- tcp <- 10.0.1.10:25
10.0.0.2:40940 -> tcp -> 10.0.1.10:25
10.0.0.2:40940 <- tcp <- 10.0.1.10:25

[MITM] Inizio sezione DATA rilevato.
10.0.0.2:40940 -> tcp -> 10.0.1.10:25
10.0.0.2:40940 <- tcp <- 10.0.1.10:25
[MITM] PAYLOAD INTERCETTATO:
Content-Type: multipart/mixed; boundary="==============8722350933425314038=="
MIME-Version: 1.0
From: attacker@attacker.local
To: victim@netsentinel.local
Subject: Test JA3

--===============8722350933425314038==
Content-Type: text/plain; charset="us-ascii"
MIME-Version: 1.0
Content-Transfer-Encoding: 7bit

Simulazione attacco
--===============8722350933425314038==--
[MITM] Fine sezione DATA rilevata.

10.0.0.2:40940 -> tcp -> 10.0.1.10:25
10.0.0.2:40940 <- tcp <- 10.0.1.10:25
10.0.0.2:40940 -> tcp -> 10.0.1.10:25
10.0.0.2:40940 <- tcp <- 10.0.1.10:25
[14:08:09.969][10.0.0.2:40940] server disconnect 10.0.1.10:25
[14:08:09.969][10.0.0.2:40940] client disconnect
luiicare@192 NetSentinel %
```

Questo assicura il fatto che il valore dell'hash calcolato sui campi del messaggio `Client_Hello` non cambia, infatti ad ogni esecuzione dello stesso script otteniamo la stessa classificazione.

Note/Sviluppi futuri

L'evoluzione dei protocolli crittografici e delle contromisure adottate dagli sviluppatori di malware, hanno reso obsoleto l'uso di JA3. In particolare attraverso il meccanismo di randomizzazione delle estensioni come GREASE nei client moderni, rende il fingerprint JA3 dinamico per il medesimo applicativo.

Ad oggi, la ricerca ha superato questo approccio monolitico, concentrandosi sull'uso di 3 metodologie principali:

1. **La Suite JA4+ (Fingerprinting Modulare)**
   Sviluppato da FoxIO, JA4+ è il successore diretto di JA3 e l'attuale standard de facto per il network fingerprinting. JA4+ utilizza un formato strutturato, decifrabile e diviso in segmenti logici (`a_b_c`):
   - **Segmento a**: Identificatore del protocollo, versione TLS e indicatore SNI.
   - **Segmento b**: Hash troncato delle Cipher Suite.
   - **Segmento c**: Hash troncato delle Estensioni e degli algoritmi di firma (ordinati alfanumericamente per resistere alla randomizzazione).

   > *Esempio di rilevazione:* Se un malware tenta di eludere i controlli modificando dinamicamente le cipher suite, il segmento `b` cambierà, ma i segmenti `a` e `c` resteranno costanti. Un analista può creare regole di rilevamento basate unicamente sul prefisso e suffisso (`JA4_ac`), rendendo inefficace la tattica evasiva.

2. **Machine Learning e Encrypted Traffic Analytics (ETA)**
   Poiché la Deep Packet Inspection (DPI) è ostacolata dalla cifratura perfetta (PFS), i moderni paradigmi di rilevazione integrano modelli di intelligenza artificiale addestrati esclusivamente sui metadati dei flussi di rete. Algoritmi di classificazione analizzano vettori di caratteristiche spazio-temporali senza necessità di decifrare il payload:
   - Sequenza e lunghezza dei pacchetti (Packet Length Sequence).
   - Tempi di inter-arrivo (Inter-Arrival Times).
   - Analisi entropica della distribuzione dei byte.
     Questo approccio permette di identificare pattern comportamentali tipici dei canali di Command & Control (C2), isolando le anomalie a livello di architettura di rete.

3. **Active Server Fingerprinting (JARM e JA4S)**

Parallelamente all'analisi del client, lo stato dell'arte prevede l'analisi attiva dell'infrastruttura dell'attaccante. Strumenti come JARM eseguono scansioni attive inviando pacchetti `ClientHello` malformati o specificamente forgiati verso server sospetti. L'analisi delle risposte

`ServerHello` permette di profilare in modo univoco l'infrastruttura backend del C2, indipendentemente dai domini temporanei o dagli indirizzi IP utilizzati.

Il sistema inoltre presenta delle limitazioni legate alla natura statica dei modelli. In particolare sono stati identificati 3 punti cruciali su cui intervenire per effettuare un'inferenza di livello superiore:

- **Sostituzione dei Modelli Monolingua:** L'utilizzo di `gpt2` e `roberta-base-openai-detector` relega il sistema all'analisi di traffico quasi esclusivamente anglofono. È necessario sostituire il classificatore discriminativo con un modello multilingua pre-addestrato su corpus eterogenei per gestire il traffico internazionale senza causare falsi positivi per dati Out-of-Distribution.

  Lo stato dell'arte prevede l'abbandono di modelli monolingua in favore di architetture come **XLM-RoBERTa** (Cross-lingual Language Model).